Cooperative Multi-Agent Navigation in Unknown Environments via Incremental Tree Repair and Shared Wall Discovery

Tianchonghui Fang[1] and Shalabh Gupta[1]

***Abstract*—Sampling-based path planners assume the map is known. Grid-based multi-agent path finding (MAPF) assumes a known graph with discrete movement. Neither assumption holds when robots enter an unfamiliar building for the first time. We extend the SMART-3D incremental replanning algorithm [1], originally designed for a single robot dodging pedestrians in a known environment, to multiple cooperating agents navigating unknown maps. Each agent discovers walls through LiDAR and repairs its RRT* tree in place: the prune-and-reconnect mechanism from the original paper, applied to permanent walls rather than transient obstacles, takes ∼5 ms versus ∼40 ms for a full rebuild. Agents share discovered walls so that one robot's LiDAR data preemptively updates all other robots' trees. A priority-based coordination layer (Global King with flee, chain push, and retreat) resolves deadlocks without touching any planning tree. Across 50,000+ benchmark trials on 8 maps (32 m to 192 m), 4 agents achieve 100% success on all large-scale scenarios. Shared discovery is $1.04\times$–$2.09\times$ faster than independent exploration. Incremental repair is $22.4\times$ faster than rebuilding from scratch. The system runs on a Raspberry Pi 5 thin client with an i9 central planner over ROS2.**

I. INTRODUCTION

The SMART-3D algorithm [1] introduced an efficient mechanism for real-time path replanning: when a moving person enters the robot's Local Reaction Zone (LRZ), it prunes affected tree edges, finds hot-nodes (tree nodes adjacent to different disconnected subtrees), and reconnects locally. A single repair costs ∼5 ms on a 15,000-node RRT* [2] tree, compared to ∼40 ms for a full rebuild. The original evaluation demonstrates this on a single robot in a fully known 2D/3D environment with randomly moving spherical obstacles.

Two assumptions limit direct deployment. First, the map is known: all static walls are loaded before planning begins. In practice, a robot entering an unfamiliar warehouse or hospital discovers walls one by one through its LiDAR. Second, the system handles one robot. Multi-robot navigation requires coordination to avoid deadlocks, especially in narrow corridors where two robots cannot pass simultaneously.

Grid-based MAPF methods [3], [4], [5] solve multi-agent coordination for hundreds of agents, but on known graphs with discrete, synchronized timesteps. Hönig et al. [6] showed that both assumptions break in real warehouses. Reactive methods like ORCA [7] handle continuous space but lack global planning and deadlock permanently in narrow corridors.

We extend SMART-3D in three directions:

1) **Unknown environments.** We add `update_static_obstacles()`, applying the same prune-and-reconnect framework to walls discovered at runtime. The key difference from the original `update()`: dynamic obstacles disappear (pedestrians walk away, edges are restored), but walls are permanent (pruned nodes are discarded, never restored). This required a new function `form_single_tree_after_static_update()` distinct from the original `form_single_tree()`.
2) **Shared wall discovery.** When robot A discovers a wall, all other robots' trees are updated immediately. This converts surprise blockages (which force expensive full rebuilds) into preemptive incremental repairs (cheap). The benefit: $1.04\times$–$2.09\times$ faster completion across 12,000 trials (Section VI-C).
3) **Reactive coordination in continuous space.** A Global King priority system with active flee, chain push, and multi-direction retreat handles deadlocks without modifying any planning tree. The mechanism is analogous to PIBT's priority inheritance [4] but in continuous coordinates rather than on a grid. 8 robots pass through a 2 m corridor at 100% success.

We validate across 8 procedurally generated maps (32 m buildings to 192 m campuses) with up to 8 agents and 30 pedestrians, totaling 50,000+ benchmark trials.

II. RELATED WORK

MAPF. CBS [3] finds optimal solutions but computation explodes in narrow corridors due to pairwise symmetry [8]. PIBT [4] scales to thousands of agents via priority inheritance. LaCAM [5] adds lazy constraints. All require a known grid and discrete timesteps. Lifelong MAPF [9] handles persistent task streams but still needs a known graph. Our system targets 2–8 physical robots in unknown, continuous environments, a regime these methods do not address.

Sampling-based replanning. DRRT [10] invalidates edges and regrows. RRTX [11] propagates cost changes globally. ERRT [12] biases toward the old tree. All require global modifications after each change. SMART-3D [1] prunes only within a local Critical Pruning Region and reconnects via hot-nodes, preserving most of the tree. We keep this local property while adding wall discovery and multi-agent coordination.

Reactive coordination. ORCA [7] computes pairwise velocity constraints. It works in open spaces but deadlocks in narrow corridors where no feasible velocity exists. DWA [13] has the same limitation. Our King/flee/push layer handles corridors by sequencing robots through priority inheritance, with the RRT* tree providing fallback paths after displacement.

III. PROBLEM FORMULATION

N agents operate in a 2D workspace $\mathcal{W} \subset \mathbb{R}^2$ with unknown static walls $\mathcal{O}_{\text{static}}$ and M dynamic pedestrians.

Agent i has position $\mathbf{p}_i$, radius r_i, speed v_i, goal $\mathbf{g}_i$, LiDAR range R_s, and a local map $\hat{\mathcal{O}}_i \subseteq \mathcal{O}_{\text{static}}$ of discovered walls. At t=0, $\hat{\mathcal{O}}_i = \emptyset$. Each agent maintains an RRT* tree $\mathcal{T}_i$ built against its current known walls.

In shared mode, agent i discovering wall w triggers $\hat{\mathcal{O}}_j \leftarrow \hat{\mathcal{O}}_j \cup \{w\}$ for all j. This models a centralized planner with shared sensor data, matching our real deployment (Section VII).

Objective: all agents reach their goals while avoiding walls, pedestrians, and each other, minimizing $\max_i t_i^{\text{arrive}}$.

IV. METHOD

The system runs a 5-phase loop every frame (Δt=0.05 s).

A. Phase 1: LiDAR Scan

Each agent casts 180 rays over 360° with range R_s. Walls behind other walls are occluded. Newly discovered walls are merged into the shared wall list.

B. Phase 2: Incremental Wall Repair

When a new wall arrives, we must decide what to do with each agent's tree. We tested five strategies over the course of development. Three survived; two were abandoned.

Full Rebuild (baseline). Throw away the 15,000-node tree, build a new one from scratch. Guarantees a clean tree, but costs 40–500 ms per wall and produces a completely different path each time. On Megacampus (128 m, ∼300 walls), this means ∼140 rebuilds per run.

Eager (E3). Apply `update_static_obstacles()`: scan all E edges, prune those intersecting the new wall, reconnect via hot-nodes within the Local Search Radius (LSR = 80 m). Cost: ∼5 ms. The tree structure is preserved.

Lazy + Eager (final). If the new wall does not block the current path, just swap the validity checker (cost: ∼0 ms). If blocked, fall back to Eager. This skips ∼80% of wall discoveries that are irrelevant to the current path.

Abandoned: Hybrid cleanup. Light edge cleanup when path unaffected. Same speed as Lazy, worse reliability. Discarded.

Abandoned: Route A (`update_new_static_obstacles`). Only check the *new* wall during Eager, not all known walls. Failed because Lazy swap leaves dirty edges from old walls; Route A never cleans them. Took 4 hours to diagnose.

Algorithm 1 describes the Eager procedure.

Detour detection. Successive repairs can degrade path quality. If $\|\text{path}\|/\|\mathbf{p}_i - \mathbf{g}_i\| > 1.5$, a full rebuild is triggered.

Why full-tree scan (E3), not path-only? An alternative ("Senior") scans only edges near the current path. This is cheaper per call but leaves dirty edges elsewhere in the tree. When an agent is pushed into those regions by another robot's King coordination, it hits invalid edges and must do a full rebuild. In multi-agent mode with shared walls, dirty edges accumulate fast: walls from remote agents affect tree regions far from the current path. E3 triggers 4–9× fewer rebuilds than Senior (Table IV), despite higher per-repair CPU cost.

Algorithm 1 Incremental Wall Repair (`update_static_obstacles`)

Require: Tree $\mathcal{T}$, new wall w, validity checker $\mathcal{V}$
$\mathcal{V} \leftarrow \mathcal{V} \cup \{w\}$
$\mathcal{E}_{\text{bad}} \leftarrow \{e \in \mathcal{T} \mid e \text{ intersects } w\}$ {$O(E)$ scan}
for each $e \in \mathcal{E}_{\text{bad}}$ **do**
  Prune e; tree fragments into subtrees
end for
$\mathcal{H} \leftarrow$ HOTNODESEARCH($\mathcal{T}$, R_{LSR})
for each hot-node $h \in \mathcal{H}$ by utility **do**
  if reconnection through h valid under $\mathcal{V}$ **then**
    Reconnect; rewiring cascade
  end if
end for
return BUILDPATH($\mathcal{T}$)

C. Phase 3: Dynamic Obstacle Avoidance

Unchanged from the original SMART-3D: compute OHZ per pedestrian, prune edges within LRZ, reconnect via hot-nodes. The King (Section IV-D) ignores other agents in planning; non-King agents treat other agents as dynamic obstacles.

D. Phase 4: Global King Coordination

This layer resolves deadlocks *without touching any planning tree*. It operates purely at the execution level: direct position adjustments via vector arithmetic, every frame.

King selection. Highest-priority active agent. Same-priority ties broken by a random value fixed at initialization. When King reaches goal, next-priority agent becomes King.

King behavior. Follows its path at full speed, ignoring all agents in planning. On collision: chain push (displace all agents along push direction). If chain hits a wall: scatter the first blocker sideways.

Non-King behavior. Within 5 m of King: flee (away, perpendicular left, perpendicular right, first non-wall direction). Beyond 5 m: follow own path, treating other agents as dynamic obstacles.

Stuck detection. No progress for 2 s triggers a rebuild. King rebuilds ignoring all agents. Non-King rebuilds with King encoded as a 3 m wall.

Three decoupled layers. (1) RRT* tree: global path via pure pursuit. (2) SMART repair: updates path when walls/OHZs invalidate edges (∼5 ms). (3) King/flee/push: direct position vector math per frame, no tree involvement. After a push, SMART replans in ∼5 ms. This is why displaced agents move smoothly in continuous space, not in grid jumps.

E. Phase 5: Shared Wall Discovery

When agent A discovers walls, the central coordinator broadcasts them to all agents. Each agent B runs `update_static_obstacles()` on its own tree. Agent B reroutes *before* physically encountering the wall.

In independent mode (ablation), each agent only uses its own LiDAR. Section VI-C quantifies the difference.

TABLE I
SCENARIO SPECIFICATIONS.

Scenario	Size	Walls	RRT* iters	Agents
building	32 m	∼41	5K	2–4
office	32 m	∼42	5K	2–4
warehouse	32 m	∼28	5K	2–4
bottleneck	32 m	∼20	5K	2–8
mega_hospital	96 m	∼200	30K	4
mega_airport	160 m	∼250	50K	4
megacampus	128 m	∼300	80K	4
mega_university	192 m	∼400	160K	4

TABLE II
SINGLE-AGENT EVOLUTION, MEGACAMPUS (1,000 TRIALS, LSR=80 M).

Strategy	Median	Rebuilds	Success	Speedup
Full Rebuild	76.3 s	139.6	96.6%	1.0×
Eager	8.7 s	2.2	99.9%	8.8×
Lazy Swap only	3.4 s	3.9	97.4%	22.4×
Lazy + Eager (E3)	best	low	100%	22.4×

V. EXPERIMENTAL SETUP

A. Scenarios

8 procedurally generated maps with walls defined as Rust functions. Agents start at corners in cross mode (navigate to opposite corner), forcing path crossings. Pedestrians follow random circular motion at 0.5–2.0 m/s.

B. Configurations

E3 (recommended): Lazy+Eager, shortcut (150 iters), detour ($\rho > 1.5$), LSR = 80 m. **Senior**: path-only repair. **Shared/Independent**: wall sharing on/off. All on an i9-14900K, 1,000 trials per scenario×config.

VI. RESULTS

A. Algorithm Evolution (Single Agent)

Table II traces the progression on Megacampus. The dominant factor is avoiding full rebuilds: 139.6 per run at baseline, 2.2 with Eager. Lazy Swap hits peak speed (22.4×) but drops to 97.4% success because dirty edges accumulate. Lazy+Eager restores 100% by cleaning all dirty edges whenever the path is blocked.

Table III confirms 100% success across all scenarios.

B. E3 vs. Senior (Multi-Agent)

Table IV shows why full-tree scan matters in multi-agent settings. Senior triggers 3–9× more rebuilds because dirty edges left by path-only repair become landmines: when King pushes an agent into an unexplored tree region, the agent hits invalid edges and must do a full rebuild (∼40 ms) instead of a cheap repair (∼5 ms). On mega_hospital, E3 succeeds in 92% of incremental repairs; Senior succeeds in 33%.

TABLE III
SINGLE-AGENT PER-SCENARIO (E3, NO PEOPLE, 1,000 TRIALS).

Scenario	Median	Success	Rebuilds
building	20.4 s	100%	0.8
office	20.6 s	100%	1.1
warehouse	14.7 s	100%	0.2
mega_hospital	56.6 s	100%	1.8
mega_airport	94.4 s	100%	2.5
megacampus	108.6 s	100%	4.9

TABLE IV
E3 VS. SENIOR, 4 AGENTS, 1,000 TRIALS EACH.

	Median (s)		Rebuilds	
Scenario	E3	Sen.	E3	Sen.
building	23.2	27.4	3.0	15.2
office	27.5	31.2	6.9	39.8
warehouse	24.2	27.2	4.1	33.4
mega_hospital	91.9	119.3	34.0	102.7
mega_airport	116.2	151.7	30.9	91.8
megacampus	117.7	121.8	54.9	121.3

E3 achieves 100% success on all scenarios. Senior drops to 99.9% on mega_hospital. CPU tradeoff: E3 wall-clock is higher (megacampus: 66.1 s vs. 39.5 s) because it scans all edges per repair. But the robot finishes faster (117.7 s vs. 121.8 s sim time). Spatial Index v2 (Section VI-F) closes this CPU gap.

C. Shared vs. Independent Wall Discovery

Table V presents the core multi-agent result. The mechanism is simple: in independent mode, a robot hits an unknown wall, its path is blocked, and it must do a full rebuild. In shared mode, another robot discovered that wall 30 seconds ago and broadcast it; the receiving robot's tree was repaired in 5 ms; it rerouted before arriving. Independent mode shows *zero* incremental repairs: without advance wall notifications, repair never triggers.

D. Bottleneck: 8 Robots, 2 m Corridor

The worst case for coordination: robots approach from both sides of a corridor too narrow for two to pass. The King system sequences them: King passes first, others flee. After King exits, next-priority agent becomes King. 100% success across all configurations (4 and 8 agents, with and without 10 pedestrians).

ORCA would deadlock here: two agents face-to-face in 2 m have no lateral space for velocity-based avoidance.

E. Deferred vs. Immediate Repair

We tested whether skipping Eager repair for walls more than 30 m away saves CPU without hurting performance. It does not help (Table VI). Deferring accumulates dirty edges; when the robot approaches, more repairs fail, triggering more rebuilds (mega_hospital: 21.6 vs. 17.9). The CPU savings (wall-clock 13.3 s vs. 16.0 s) are offset by extra rebuilds. Immediate Eager is optimal.

F. Spatial Index v2

E3's cost is dominated by scanning all E edges per new wall. We partition the workspace into 4 m×4 m cells and

TABLE V
SHARED VS. INDEPENDENT, 4 AGENTS, E3, 1,000 TRIALS PER CELL.

Scenario	Shared	Indep.	Speedup
building	23.2 s	48.4 s	2.09×
office	27.8 s	33.5 s	1.21×
warehouse	24.3 s	26.9 s	1.10×
mega_hospital	92.8 s	108.5 s	1.17×
mega_airport	115.8 s	121.0 s	1.04×
megacampus	116.8 s	137.4 s	1.18×

Per-agent fairness: on megacampus, the gap between slowest and fastest agent drops from 12.3 s (independent) to 2.9 s (shared), a $4.2\times$ reduction. Office: $4.4\times$. Mega_hospital: $2.5\times$.

TABLE VI
IMMEDIATE VS. DEFERRED EAGER (30 M), 4 AGENTS, 1,000 TRIALS.

Scenario	Immediate	Deferred	Verdict
building	23.1 s	23.3 s	Same
office	28.1 s	27.9 s	Same
mega_hospital	92.0 s	94.9 s	Imm. better
mega_airport	115.3 s	119.8 s	Imm. better
megacampus	116.6 s	115.6 s	Same

register each edge in the cells it passes through. Per new wall, only edges in overlapping cells are checked. The pruned set is identical to full scan: an edge far from a wall cannot intersect it.

v1 (heap-allocated `Vec` per query) was *slower* than linear scan: 300 walls fit in 4.8 KB (L1 cache); linear scan has perfect prefetch. v2 eliminates all allocation: allow duplicate checks (5 ns overhead), single-cell fast path. Megacampus: 66.1 s $\rightarrow$ 48.6 s (-26%). Small maps slightly slower (HashMap overhead), irrelevant at <1 s total.

VII. REAL ROBOT DEPLOYMENT

Hardware. Raspberry Pi 5 (4-core, 2.4 GHz) runs Cartographer SLAM and motor control. Alienware i9-14900K runs SMART-3D in Rust. CycloneDDS over WiFi, ROS_DOMAIN_ID=42.

Why centralized. Cartographer on Pi 5 consumes $\sim$1.5 cores. RRT* takes $\sim$300 ms on Pi 5 vs. $\sim$40 ms on i9. Running both on-board is not feasible.

Sensing. Pi 5 streams LiDAR scans and SLAM pose to i9. i9 extracts walls from the occupancy grid, classifies LiDAR points not near any known wall as dynamic, clusters them (min 8 points, 1.0 s persistence), tracks velocity, feeds (x, y, r, v) to the Rust planner.

Multi-agent. One Rust subprocess per agent with per-agent `robot_radius`. Other agents encoded as dynamic obstacles with OHZ.

VIII. DISCUSSION

Scalability. Per-frame: $O(N^2)$ pairwise checks, $O(N \cdot E)$ shared repair. Spatial index reduces the latter to $O(N \cdot k)$, $k \ll E$. Tested up to 8 agents.

Limitations. King handles King-vs-non-King deadlocks. Two non-King agents in a narrow corridor have no priority differentiation; mega_hospital shows 96–100% success depending on config. Extending priority to non-King pairs would fix this.

TABLE VII
SPATIAL INDEX V2: CPU TIME PER TRIAL (1,000 TRIALS).

Scenario	Before	After	Change
building	0.5 s	0.7 s	+40%
mega_hospital	13.4 s	11.2 s	−16%
megacampus	66.1 s	48.6 s	−26%

What we borrowed. From the SMART-3D crate [1]: `SMART::update()` (dynamic obstacles, unchanged), `RRTstar::new()`, `hot_node_search()`, `build_planned_path()`.
What we built: `update_static_obstacles()` with `form_single_tree_after_static_update()`, Lazy+Eager architecture, Spatial Index v2, Global King coordination, shared wall discovery, 8 scenario maps, deployment stack, 50,000+ trial benchmarks.

IX. CONCLUSION

We extended SMART-3D from a single-robot, known-environment replanner to a multi-agent system for unknown maps. Incremental repair is $22.4\times$ faster than rebuilding. Shared wall discovery is $1.04\times$–$2.09\times$ faster than independent. The King layer passes 8 robots through a 2 m corridor at 100%. All over 50,000+ trials and deployed on a physical robot with ROS2.

REFERENCES

[1] P. Agrawal, S. Gupta, and Z. Shen, "SMART-3D: Three-dimensional self-morphing adaptive replanning tree," 2025. [Online]. Available: https://arxiv.org/abs/2509.16812

[2] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011. [Online]. Available: https://doi.org/10.1177/0278364911406761

[3] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015. [Online]. Available: https://doi.org/10.1016/j.artint.2014.11.006

[4] K. Okumura, M. Machida, X. Défago, and Y. Tamura, "Priority inheritance with backtracking for iterative multi-agent path finding," *Artificial Intelligence*, vol. 310, p. 103752, 2022. [Online]. Available: https://doi.org/10.1016/j.artint.2022.103752

[5] K. Okumura, "LaCAM: Search-based algorithm for quick multi-agent pathfinding," in *Proc. AAAI Conference on Artificial Intelligence*, 2023. [Online]. Available: https://arxiv.org/abs/2211.13432

[6] W. Hönig, S. Kiesel, A. Tinka, J. W. Durham, and N. Ayanian, "Persistent and robust execution of MAPF schedules in warehouses," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1125–1131, 2019. [Online]. Available: https://doi.org/10.1109/LRA.2019.2894217

[7] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research*. Springer, 2011, pp. 3–19. [Online]. Available: https://doi.org/10.1007/978-3-642-19457-3_1

[8] J. Li, D. Harabor, P. J. Stuckey, H. Ma, and S. Koenig, "Symmetry-breaking constraints for grid-based multi-agent path finding," in *Proc. AAAI Conference on Artificial Intelligence*, 2019. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/4565

[9] J. Li, A. Tinka, S. Kiesel, J. W. Durham, T. K. S. Kumar, and S. Koenig, "Lifelong multi-agent path finding in large-scale warehouses," in *Proc. AAAI Conference on Artificial Intelligence*, 2021, pp. 11 272–11 281. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/17344

[10] D. Ferguson, N. Kalra, and A. Stentz, "Replanning with RRTs," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2006. [Online]. Available: https://doi.org/10.1109/ROBOT.2006.1641879

[11] M. Otte and E. Frazzoli, "RRTX: Asymptotically optimal single-query sampling-based motion planning with quick replanning," *International Journal of Robotics Research*, vol. 35, no. 7, pp. 797–822, 2016. [Online]. Available: https://doi.org/10.1177/0278364915594679

[12] J. Bruce and M. Veloso, "Real-time randomized path planning for robot navigation," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2002. [Online]. Available: https://doi.org/10.1109/IRDS.2002.1041624

[13] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics and Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997. [Online]. Available: https://doi.org/10.1109/100.580977